Linus Mika Brandt: 108021255292
Maximilian Frese: 108021213184
Frederik Hüttemann: 108021215247
Christopher Robin Thomas: 108021211653
Miriam Fee Unger: 108021203897

Sprint Review and Retrospective – Summary

Sprint Review

The first sprint ran from the Sprint Planning on the 27th of April until the Sprint Review on the 10th of May. Our goals for the sprint were formulated, as recommended by our group advisor Ahmad, relatively broadly. His word of advice was that during and after the first sprint, the app would quite probably not show much improvement or change in relation to how much work we put in, but the underlying framework would facilitate the development process in the coming sprints greatly.

This prediction proved quite accurate. Many of the distributed tasks were prerequisites for later tasks and user stories, such as getting access to sensor data, interacting with Android's internal storage, sending notifications, and providing an interactable app framework onto which functionality can later be built. For many of these, it was necessary to communicate with the Android System and Android Studio on varying levels of depth and a decent chunk of time was spent on researching how these features can best be implemented. While this time was not spent on measurable progress, we calculated with this in mind, distributed relatively high-level tasks and are sure that the knowledge gained here will come in handy later on in the project.

The tasks which we distributed at the start of the sprint were all achieved. We did plan with a solid time buffer and could probably have implemented a few more features and maybe even made additional tangible progress before the next sprint, but as described, we used the time well anyway.

Sprint Retrospective

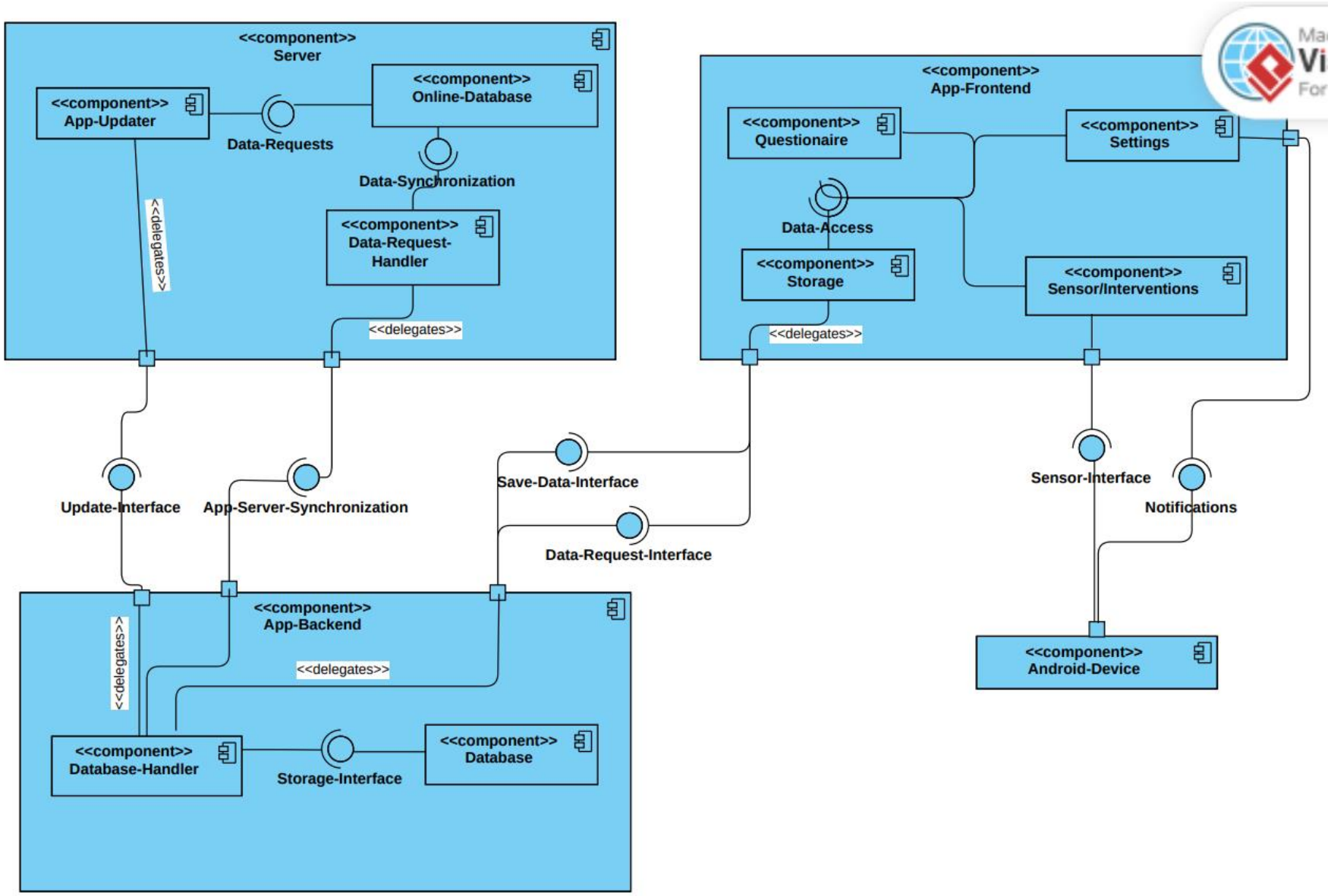
Before the sprint, we agreed, partially as a result of the first task in the previous group assignment, on a few basic guidelines for our group work. This included two weekly meetings, online on Mondays and at the university on Thursdays, a clear distribution of work as mentioned before and the creation of a discord server for micro-communication, announcement and similar messaging. We were aware that our communication would be limited in the beginning, as everyone needs to first immerse themselves in their area of specialty before they can start interacting and sharing. Thus, apart from the meetings, which worked quite well for syncing progress and were all attended by all group members except one which fell on a public holiday, everyone worked in relative isolation.

While this may also just be because not much help was needed and the tasks were clear and decision-free, we all agreed in the sprint retrospective that we want more communication. We want to combine an increase of work in pairs or groups with a bit of a shift in our usage

of GitHub. The branching system served us well for the most part, as everyone was able to implement their features free of interference and conflicts, but the large delay between starting a task and merging it into the main branch prevented effective collaboration and resulted in too much isolation. Therefore, we concurred that we need to commit more frequently and use branches for somewhat smaller parts of features.

Moving away from the few negative aspects, our group atmosphere was stellar. We were very effective in our meetings, agreed quite quickly on important decisions and did not struggle with keeping up with the project's progress. Everyone seemed very eager to complete their tasks and contributed to our work. The climate was so good, in fact, that during the sprint retrospective, we agreed that sometimes a bit more focus and seriousness is needed during the meetings because we had too much fun joking around with each other when we could have been more effective and disciplined.

All in all, the first sprint was a positive experience. We planned well, set systems in place which will facilitate future work and executed on that planning effectively. We are looking forward to continue working on the second sprint with slight adjustments to our workflow and continuing enthusiasm.


<<component>>
Server
<<component>>
App-Updater
Data-Requests
<<component>>
Online-Database
Data-Synchronization
<<component>>
Data-Request-
Handler
<<delegates>>
<<delegates>>
<<component>>
App-Frontend
<<component>>
Questionaire
<<component>>
Settings
Data-Access
<<component>>
Storage
<<component>>
Sensor/Interventions
<<delegates>>
Update-Interface
App-Server-Synchronization
Save-Data-Interface
Data-Request-Interface
Sensor-Interface
Notifications
<<component>>
App-Backend
<<delegates>>
<<delegates>>
<<component>>
Android-Device
<<component>>
Database-Handler
Storage-Interface
<<component>>
Database

The presented component diagram sketches our current app design. At the moment, we are planning to use three main components. The first one will the app itself being the frontend with all the different activities as components. Additionally, we need the local backend which deals with data storing and as well as with synchronising the local storage with the server-sided database. The server will also store data especially the data we want to provide to the customer.

The App itself has three main components and one storage component. The storage component will handle all the data access requests by the three other components of the app. This includes requesting data and saving data. For this the storage component provides an interface. Currently this interface is realized in a class called “DBHelper”. When any of the app’s components want to save or access data the DBHelper interface is used. For the DBHelper we used the Singleton pattern.

Hence to realize the user-stories about the recording of the effective wellbeing, the questionnaire component will record the answers from the user and then uses the Data-Access Interface. Same goes with the sensor/intervention’s component. When the user wants to record an intervention and this includes sensor data, the saved data will also be stored using the same interface. To record sensor data the sensor interface provided by the android device itself is used.

The app’s backend is designed with two sub-components. One is the database itself and one being the database-handler which handles requests and also deals with synchronization with the server. The database handler provides two interfaces for the app-frontend. One for data-request, one for saving data. The Database Handler then always saves the newly incoming data in the local database using the storage interface provided by the database. When the frontend requests data from the backend, the database handler then access the locally saved data from the database and provides it to the frontend.

Apart from the frontend-backend communication, the database-handler also communicates with our server. Here the main tasks are updating the local data and uploading monitored sensor data and questionnaire questions to provide this information for the customer.

The App-Server-Synchronization will use IDs to make the communication and information pseudonymized. The main part of this communication will be the upload of the recorded data of the user which has been monitored during interventions or when the user decided to the app to record the user’s data. Additionally, the local Database helper can use the by the server provided interface to check if there are possible updates, a new algorithm or new questions which should be then added to the questionnaire component.

The reason why we chose this architecture is mainly because the app needs to work offline while the researcher should still be able to access data collected by the app.

They way we want to archive this duality is by having two databases. One being managed locally on the users’s device and one database on the server which will synchronize every time the user is reconnecting to the internet. Having one database locally which saves all the data we ensure that will can always show our data visually and the user can always answer the questionnaire no matter if he is connected to our server or not.

Because we also upload our data anonymously to our server, we can provide the research our collected data easily. Additionally, we have a way to communicate with all user's at the same time when new features need to be uploaded.

This way we can implement all the requirements dealing with the data collection. Additionally, we ensure that interventions will be proposed as intended and all recordings will also be saved locally and later online for further analysis. Because the user should be able to see all interventions he recorded we save those data locally as well.